

Input Embeddings — Transformer Accelerator

Testing pipeline for a single self-attention block

Dataset — Tiny Shakespeare

Using Karpathy's tinyshakespeare (~1.1 MB, ~1.1M chars) for next-word prediction testing only. No training. A 512-token window is enough to test the embedding path.

github.com/karpathy/char-rnn/blob/master/data/tinyshakespeare/input.txt

Tokenization

GPT-2 uses BPE (Byte-Pair Encoding) with a vocab of 50,257 tokens. Each word/subword maps to a token index. Try it at: platform.openai.com/tokenizer

```
import tiktoken
enc = tiktoken.get_encoding('gpt2')
enc.encode('Hello')  # → [13225]
```

Text	Token ID
"Hello"	13225
"First"	5962
" we"	356 (space is part of the token)

Embedding Matrix — GPT-2 wte

Load the trained matrix from HuggingFace. It's a lookup table — one row per token.

```
from transformers import GPT2Model
model = GPT2Model.from_pretrained('gpt2')

model.wte.weight.shape  # torch.Size([50257, 768])
model.wte.weight.dtype  # torch.float32 (~147 MB)
```

Getting the Input Embedding

Token ID → row lookup in wte. For a sequence of S tokens, batch it into a 3D tensor:

```
embedding = model.wte.weight[13225]  # shape: [768]

# For a full sequence (B=4, S=512):
x = model.wte(token_ids)  # shape: [4, 512, 768] ← input to self-attention
```

Validation — OpenAI Tokenizer == GPT-2 Matrix?

Both tiktoken and HuggingFace's GPT2Tokenizer use the same vocab.json and merges.txt, so token IDs are identical. Confirmed — safe to use tiktoken to get indices for wte lookups.

```
enc_oai = tiktoken.get_encoding('gpt2')
enc_hf = GPT2Tokenizer.from_pretrained('gpt2')

enc_oai.encode('Hello') == enc_hf.encode('Hello')  # True
```

Numerical Representation — FP32 / FP16 / INT16

```
emb_fp32 = model.wte.weight[13225]  # default float32
emb_fp16 = emb_fp32.half()          # float16

scale = emb_fp32.abs().max()
emb_int16 = (emb_fp32 / scale * 32767).round().to(torch.int16)  # INT16
```

Format	Bytes/vector	wte size	Error vs FP32
--------	--------------	----------	---------------

FP32	3072 B	~147 MB	none (baseline)
FP16	1536 B	~73 MB	~5e-4
INT16	1536 B	~74 MB	~2e-5 (better than FP16)
INT8	768 B	~37 MB	~5e-3

INT16 is actually more accurate than FP16 here — GPT-2 embeddings are small values ($|x| < 1.5$) and FP16's 10 mantissa bits introduce more rounding error than a scaled INT16.

Hardware Takeaway

Embedding lookup is memory-bound — just indexed reads, zero compute. The 147 MB wte matrix must stream from HBM. Storing in INT16 halves bandwidth with almost no accuracy cost. Recommended: store embeddings as INT16, upcast to FP16 after lookup, run attention in FP16.

text → token ID → wte[token_id] → 768-dim vector → [B, S, 768] tensor → self-attention